

面向边缘智能的 AIOS 设计与优化

决赛进展汇报

以一个真实的边缘 AI 机器人为牵引，将 StarryOS 在 RK3588 上做到对 Linux 逐轴对等乃至反超，
成果全部回馈上游 rcore-os/tgoskits（累计 90 个 PR 已合并）。

Team Posad · 清华大学

RK3588 / OrangePi-5-Plus · 基线 rcore-os/tgoskits dev @73409e079 · 2026-08

网球机器人与拾球回路



- ▶ 闭环起于相机成帧，依次经 JPU 解码、RGA 预处理、NPU 推理，直至定位与运动控制，再返回相机
- ▶ 帧到指令延迟即闭环运行一周的端到端时间
- ▶ 追球交由 NPU 上的单类 YOLOv8，寻桶采用整型 HSV 红色掩膜
- ▶ 感知结果送入纯逻辑状态机，生成底盘与舵机指令

从初赛到决赛：从机器人推理到整个操作系统

初赛

- ▶ 把网球拾取机器人的操作系统由 C 语言的 Linux 迁移到 Rust 实现的 StarryOS
- ▶ 优化机器人的视觉推理链路，使端到端推理在真机上追平 Linux（下页详述）

决赛

- ▶ 命题扩展为整机对等：让整个 StarryOS 作为通用 OS，在调度、内存、频率、启动等子系统上也对 Linux 逐轴对齐
- ▶ 机器人成为众多负载之一，考核操作系统的通用能力

工作量以回馈上游 `rcore-os/tgskits` 计量（自初赛至今累计；决赛期新增 24 已合并 + 19 在审）：

90

PR 已合并

122

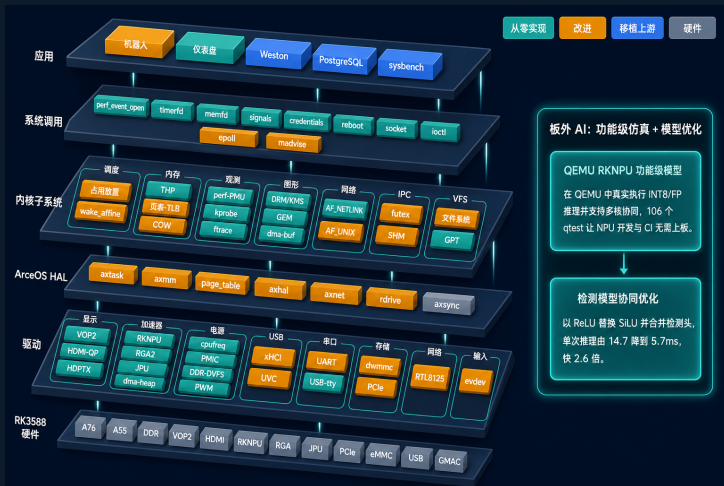
PR 累计提交

19

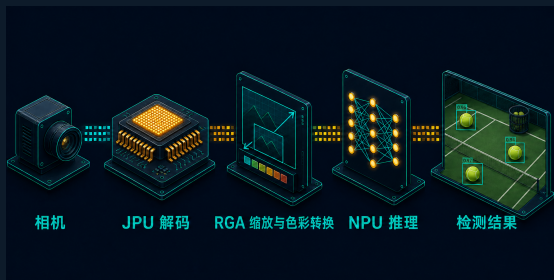
PR 评审中

覆盖系统调用/进程/信号、调度/大小核、内存/分页、观测 `perf`、RGA/JPU/NPU 与显示驱动、网络、虚拟化、板级；另自建 QEMU RKNPU 功能级模型，全部经 RK3588 真机验证。

自底向上：本队贡献贯穿整个软件栈



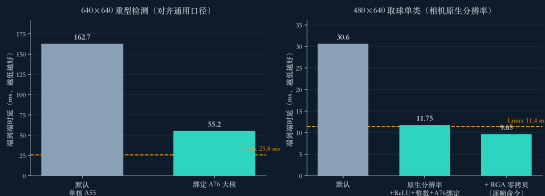
感知流水线：三块自研加速器驱动 + dma-buf 零拷贝



- ▶ **JPU** 硬件 JPEG 解码：新增 `/dev/mpp_service`，对齐瑞芯微 MPP；单帧 1ms、与 Linux 逐字节一致（#1456）
- ▶ **RGA** 缩放与色彩转换：新增 `/dev/rga` RGA2 驱动，对齐 librga，较 CPU 快 7.4 倍（#1388）
- ▶ **NPU** 连续推理：修正 GEM 缓冲缺页与引用计数，逐帧稳定、首帧与 Linux 一致（#1364）
- ▶ **零拷贝** 三块加速器经 dma-buf 共享物理内存，整条流水线全程零拷贝

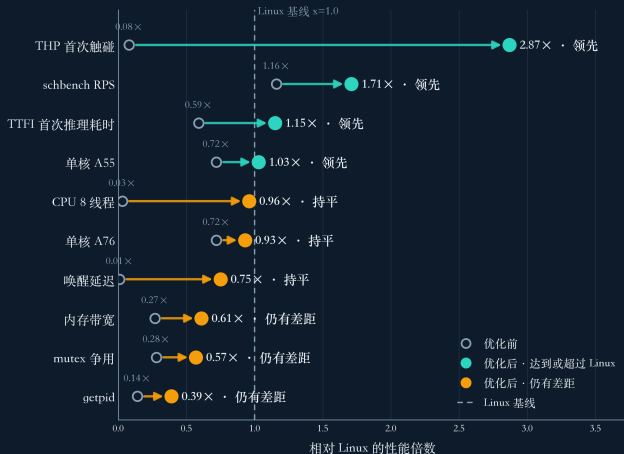
视觉推理链路：重型模型 $6.3\times$ 收窄至 $2.1\times$ ，小模型追平 Linux

端到端时延阶梯：两种输入尺寸各自与同尺寸 Linux 基线比较



- ▶ 同一份 ELF 两侧对测，StarryOS 起点端到端落后 Linux 约 6.3 倍，热点散在链路各阶段
- ▶ 绑定 A76 大核： 640×640 端到端 $162.7\rightarrow 55.2$ ms，三大 CPU 阶段各降约 3 到 5 倍
- ▶ 整数取回路径 (`want_float=0`) 使 `outputs_get` 约 $12\rightarrow 1$ ms，285 张测图逐张一致
- ▶ 原生 ReLU 模型将 480×640 p50 $30.6\rightarrow 11.75$ ms 追平 Linux，再加 RGA 零拷贝逐帧命令降到约 9.65 ms

跨子系统对 Linux 的性能追平：优化前 → 优化后



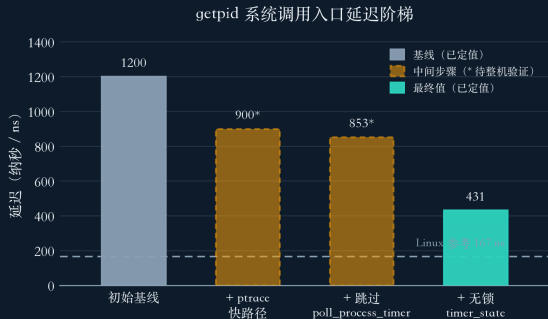
注：各项均为 RK3588 整机实测，3 次重复取中位数；吞吐类为 Starry/Linux，时延类为 Linux/Starry

对 Linux 的完整基准结果 (StarryOS 优化前 → 后)

轴 (单位)	优化前	优化后	Linux	判定
THP first-touch (s)	0.8 到 1.3	0.030	0.086	领先 2.87×
启动 TTFI (s)	19.22	9.83	11.28	领先
A55 单核 (ev/s)	约 251	359	349	领先 1.03×
schbench RPS	231	339	199	反超 1.71×
A76 单核 (ev/s)	约 693	896	962	持平 0.93×
cpu 八线程 (ev/s)	160	4987	5222	持平 0.96×
唤醒 p50 (μs)	1042	8	6	持平
getpid (ns)	1200	431	167	收窄 2.6×
mutex 8T (s)	1.69	0.85	0.481	收窄 1.76×
内存写 (MiB/s)	15040	34330	56046	收窄 0.61×

“收窄”指相对 Linux 的差距已大幅缩小 (getpid 由 7× 收至 2.6×, mutex 由 3.5× 收至 1.76×); 内存写受 DDR 变频固件限制。hackbench 的 -T /-P 比值经无锁用户拷贝快路径恢复到约 1 (回到 Linux 形态)。

系统调用入口：getpid 微阶梯

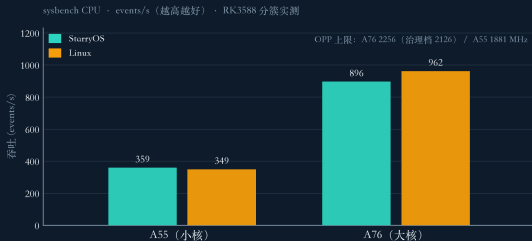


* 初始基线与中间两级为实现当期单次测量、最终值与 Linux 参考取三次重复中位数

- ▶ 空系统调用 getpid 基线 1200 ns, Linux 167 ns, 相差约 7 倍, 开销集中于进入与返回路径
- ▶ 三层剥离每次调用均付而多数不用的固定成本: ptrace 快路径
– 25%、跳过 poll_process_timer
– 17%、无锁 timer_state 粗 tick 记账 – 50%
- ▶ getpid 降至 431 ns, 相对 Linux 由 7 倍收窄至 2.6 倍, stime 秒占比 0.71 对齐 Linux 0.72 (残余为 uctx.run 陷入的架构地板)

频率电压：DVFS / PVTPLL 与 DDR 变频固件外因

DVFS 分核性能追平 Linux



- ▶ RK3588 CPU 时钟为电压耦合的 PVTPLL，频率随轨压变化，CPU CRU 由 BL31 保护
- ▶ #1657: 按簇 ondemand governor + PMIC 电压标定 (A76 走 I2C、A55 走 SPI)，事务式先电压后时钟
- ▶ 环长调节: 更高 SCMI 速率将环由 17 重编为 11，同压更高频; A76 顶 2126 MHz、A55 顶 1881 MHz，八线程达 $0.96 \times$ Linux
- ▶ **DDR 变频**驱动已实现，因主线 TF-A 缺 SIP DRAM 的 EL3 处理器而停在探测态 (外因)

观测能力：硬件 PMU perf 与 ftrace

- ▶ 前页的对等结论均建立在测量之上，而测量依赖工具；决赛前 StarryOS 尚无基于硬件 PMU 的 perf，热点难以定位到层
- ▶ 构建一整套硬件 PMU perf：单核 PMUv3、big.LITTLE 分簇、软件与 cache 事件、事件组、调用栈、kprobe/tracepoint、ftrace
- ▶ 未经修改的上游 perf 6.1.0 二进制直接产出 perf.data、report 与 top；板级 big.LITTLE 矩阵 39/0、特性矩阵 56/0
- ▶ 受硬件限制的 PEBS 与 BRBE 类特性，按 Linux 在同一 SoC 上的行为如实拒绝，未伪造读数

ftrace 全量函数跟踪在真实 8 核开发板活锁（QEMU 因 smp1 未暴露），过滤式函数跟踪为板上与 QEMU 均实用的模式。

perf 真机实测：硬件 PMU stat / record / report

```

root@starry:~# perf --version
perf version 6.6.0
root@starry:~# perf stat -e cycles,instructions,cache-references,cache-misses /usr/bin/true

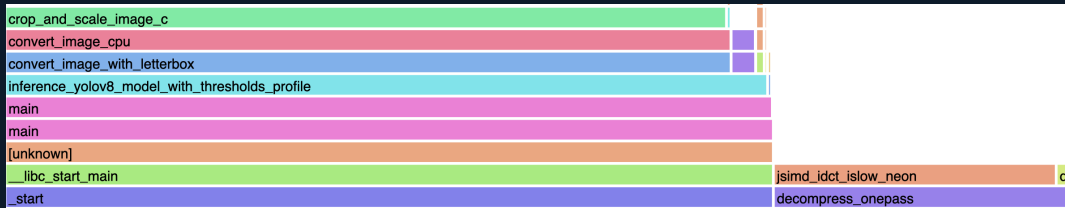
Performance counter stats for '/usr/bin/true':

    35995945      armv8_cortex_a55/cycles/                (96.57%)
    35968256      armv8_cortex_a76/cycles/                (97.00%)
    35940436      armv8_pmu3_0/cycles/                    (98.63%)
    27795620      armv8_cortex_a55/instructions   #    0.77  insn per cycle (99.25%)
    27767937      armv8_cortex_a76/instructions   #    0.77  insn per cycle (99.32%)
    27733787      armv8_pmu3_0/instructions       #    0.77  insn per cycle
    6609084       armv8_cortex_a55/cache-references      (3.43%)
    22653         armv8_cortex_a55/cache-misses  #    0.34% of all cache refs
    19724         armv8_cortex_a76/cache-misses  #    0.30% of all cache refs

    0.122275417 seconds time elapsed
root@starry:~# perf record -F 299 -a -- sleep 2
[ perf record: Captured and wrote 0.061 MB /root/pf.data (1062 samples) ]
root@starry:~# perf report --stdio
# Overhead  Command  Shared Object  Symbol
   39.41%   perf-exec [kernel.kallsyms] ax_kernel_guard::NoPreemptIrqSave::release
   19.07%    perf [kernel.kallsyms] memcpy
   18.99%    sh [kernel.kallsyms] ax_kernel_guard::NoPreemptIrqSave::release
    1.98%    perf [kernel.kallsyms] ax_task::api::run_idle
    1.71%    perf [kernel.kallsyms] KernelGuardIf::enable_preempt
root@starry:~#

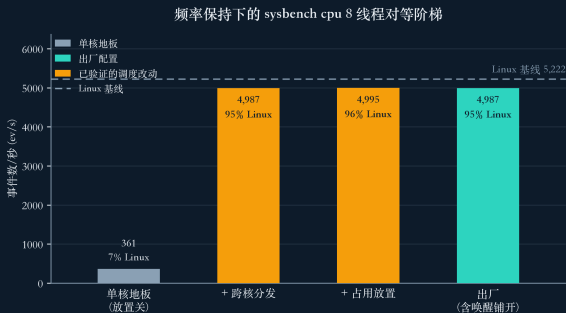
```


perf 火焰图：网球推理的 CPU 调用栈



perf record -g 采样 6735 次，符号经 /proc/kallsyms 解析——完整用户态调用栈 `_start` → `main` → `inference_yolov8` → `convert_image` / `crop_and_scale`，右支为 libjpeg 的 NEON 解码 `jsimd_idct_islow_neon`

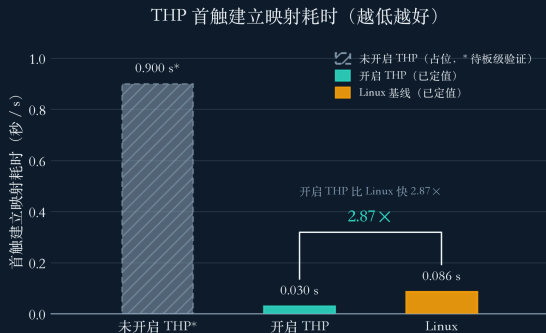
调度：sysbench cpu 对等阶梯



注：单独启用 wake_affine 就近唤醒会因并发线程聚拢而将 8 线程吞吐降至 3523，出厂配置的唤醒铺开将其恢复至 4987。

- ▶ 决赛初整机压在单个 A55，八线程仅 160 ev/s，落后 Linux 约三十倍（下图为频率固定下的隔离阶梯，地板 361）
- ▶ 固定顶频以隔离放置层：跨核分发（#1656）把八线程由单核地板 361 抬到 4987 ev/s，为最关键一层
- ▶ 叠加占用感知放置续进至 4995 ev/s，增益已属小幅；出厂配置八线程 4987 ev/s，为 Linux 5222 的 0.96 倍，判定持平

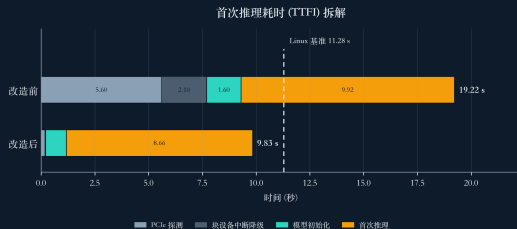
内存：THP first-touch (领先)



* 未开启 THP 为占位值, 等待整机 ablation 复测确认 (不影响开启 THP / Linux 两侧已定值)

- ▶ 首触路径慢: 128 MB fault-in 需 0.8 到 1.3 s, Linux 0.086 s, 逐页约 9 到 15 倍
- ▶ 三级压缩: 新建映射跳过广播 TLB flush、aarch64 dc zva 块清零、THP-lite 2 MB 私有匿名提升
- ▶ 128 MB 私有匿名首触 (2 MB THP-lite 提升) 降至 0.030 s, Linux 0.086 s, 快 2.87 倍 (领先)
- ▶ 收益来自硬件层面 (更少 TLB 广播、块清零、每 2 MB 少 512 次缺页), Linux 侧缺乏等价用户态手段

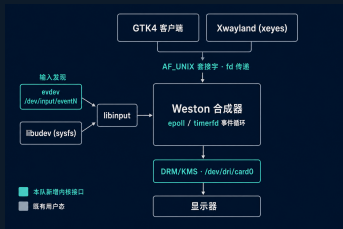
启动：首帧推理 TTFI（领先）



各阶段切分为估算值，待整板复测确认更新（改造前/改造后/Linux 总耗时已由串口时间戳板测确认）

- ▶ 口径：内核自 U-Boot 接管起至 live 首帧 RKNN 推理完成
- ▶ 提交版本 19.22s 慢 Linux 约 1.7 倍；经根因定位与约十一处公平改动降至 9.83s，优于 Linux 11.28s（领先）
- ▶ 内核侧两段：PCIe 空槽串行探测约 5.6s（次核前置 + FDT 借用 + 空槽快返），块 IO 完成中断回退约 2.1s
- ▶ 应用侧最大一段：相机边流边初始化使 rknn_init 饥饿，先加载模型再启相机将 model_init 由 16870 降至 140 ms

显示、图形与板级：从零打通的广度



- ▶ **原生显示栈** 从零实现 VOP2 + HDMI-QP + HDPTX 四个 `#![no_std]` crate, Qt6 时钟经 `/dev/fb0` 上屏; **上板点亮验证通过** (ROPLL 锁定、1080p60 光栅、彩条、Qt6 运行)
- ▶ **Weston 合成器** 在 StarryOS 上运行 Weston: 补齐 `/dev/dri/card0` 原子 KMS、多路输入、AF_UNIX 传 fd、epoll 就绪语义, 四架构验证, 图形栈 #503-516 / #1160 已合并
- ▶ **板级 I/O 与网络** UART、USB-serial tty、PWM、UVC 相机、reboot(2) 与网络驱动均合入上游; pinctrl 修正 + RTL8125 + rtnetlink 打通板上 SSH, 暖重启回路秒级迭代

The screenshot displays a Linux desktop with a dark-themed window manager. In the background, a terminal window titled "Wayland Terminal" is open, showing a shell prompt and several commands being executed: `ls gtk4-demo`, `gtk4-demo 40;`, `gtk4-wayland --`, `gnome s settings org.gnome.desktop.interface gtk-theme gtk4`, `gnome s ping -c 4 cs`, and `gnome s ping -c 4 cs`. The output of the last command shows network statistics: 4 packets transmitted, 3 packets received, 25% packet loss, and round-trip times ranging from 1059.727 to 1093.213 ms. In the foreground, a window titled "GTK Demo" is open, displaying a simple web browser interface with a search bar and a list of search results. The search results include "GTK Demo is a cool and feature-rich" and "You can select your search term. Check demo, the source code is in the area." The desktop also shows a taskbar at the bottom with icons for the application menu, file manager, and other utilities.

Weston 合成器 + GTK4 / Xwayland, 四架构验证通过

QEMU RKNPU 设备模型与机器人检测模型的协同优化

```
t_type=AFFINE, zp=-128, scale=0.003173
  index=3, name=338, n_dims=4, dims=[1, 64, 40, 40], n_elems=102400, size=102400, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-45, scale=0.093747
  index=4, name=onnx::ReduceSum_346, n_dims=4, dims=[1, 80, 40, 40], n_elems=128000, size=128000, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003594
  index=5, name=350, n_dims=4, dims=[1, 1, 40, 40], n_elems=1600, size=1600, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003627
  index=6, name=357, n_dims=4, dims=[1, 64, 20, 20], n_elems=25600, size=25600, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-34, scale=0.083036
  index=7, name=onnx::ReduceSum_365, n_dims=4, dims=[1, 80, 20, 20], n_elems=32000, size=32000, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003874
  index=8, name=369, n_dims=4, dims=[1, 1, 20, 20], n_elems=400, size=400, fmt=NCHW, type=INT8, qnt_type=AFFINE, zp=-128, scale=0.003922
model is NHWC input fmt
model input height=640, width=640, channel=3
init_yolov8_model success!
Model info: width=640, height=640, channel=3
convert image use cpu
inference_yolov8_model success!
=== Detection Results Summary ===
Detected objects: 1
[1] sports ball
    Confidence: 70.8%
    Position: (479, 706, 773, 1010)
    Box Size: 294x304
    Center: (626, 858)
UVC_RKNN_IMAGE_DONE
RK3588-RKNPU-TEST workload exit=0
[ 6.175117] vendor storage:20190527 ret = -1
```

- ▶ 本队将闭源 RK3588 RKNPU 实现为功能级 QEMU 模型 (qemu#19、qemu#26, 106 qtests), 真实执行 INT8/FP 并接入 IOMMU
- ▶ 模型将板上调试的加速器转为 board-free、寄存器精确的开发、CI 与优化载体
- ▶ 在该模型上逐层重塑网球检测模型: SiLU 改 ReLU 使 LUT 任务 107→3、rknn_run 减半, 叠加类别头收窄与头框合并

借鉴与创新：基础版本与本队增量

站在何处（借鉴的基础版本）

- ▶ 内核：rcore-os/tgoskits 的 dev 分支 @73409e079 (StarryOS + ArceOS 组件)
- ▶ 机器人应用：移植自 pengzechen/aka-rk3588
- ▶ 模型转换：airockchip/rknn-toolkit2
- ▶ 每处改动首次提交即固定基线、如实标注来源

本队的创新与增量工作

- ▶ 机器人视觉推理链路：端到端由落后 Linux 约 6.3 倍优化到持平
- ▶ 整 OS 对 Linux 逐轴对等，THP / TTFI / A55 / schbench 领先；全部拆为原子 PR 回馈上游，累计 90 个已合并、19 个在审，覆盖调度/内存/观测/驱动/显示/网络/虚拟化等子系统
- ▶ **创新** QEMU RKNPU 功能级设备模型：把闭源 NPU 变为板外可开发、可 CI 的载体（106 qtest）
- ▶ **创新** AI 掌控-闭环开发方法 + 带外电源自治测试台

小结、分工与材料

- ▶ **贡献小结** 机器人推理追平 Linux；整 OS 对 Linux 逐轴对等，THP / TTFI / A55 / schbench 领先；QEMU NPU 设备模型与 AI 开发范式；全部成果回馈上游 rcore-os/tgoskits，累计 90 个 PR 已合并、19 个在审
- ▶ **分工** 洪世金（队长，内核性能/内存/驱动/显示）、王乙凡（QEMU NPU 模型 + 机器人应用）、徐子航（板级 I/O 与网络 + 机器人应用）
- ▶ **后续** 调度全量均衡、DVFS Phase 2 电压闭环、DDR SIP 换 rkbin BL31、THP 通用化、NPU 三核上真机
- ▶ **材料与复现** 同一份 ELF 两侧对照、三次重复中位数；代码见 rcore-os/tgoskits 与 processmission/qemu，报告与幻灯见 docs/final-round/

附录 · 备用材料

目标完成表 · 基准与口径 · 增量 PR 一览 · 与类似工作对比 · 机器人状态机 ·
视觉推理链路 · AI 使用说明

系统分层总览与术语



- ▶ StarryOS 复用 ArceOS 模块层，补齐 Linux 兼容的系统调用、进程与信号，按设备类型分层驱动
- ▶ 本队改动集中于调度、DVFS、分页、观测四层，另含若干 RK3588 驱动

术语一次讲清：

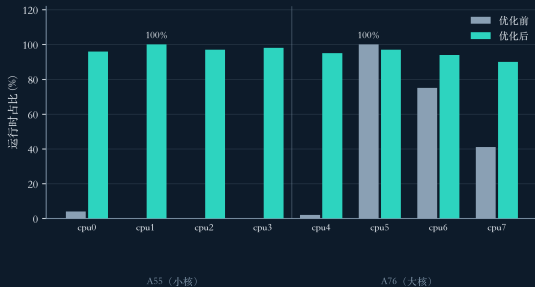
big.LITTLE = 4×A55 小核 (cpu0-3) + 4×A76 大核 (cpu4-7)；

DVFS = 动态调频调压；THP = 透明大页；

TTFI = 上电到首帧推理。

调度：核驻留与唤醒延迟

核心占用率：负载均衡优化前后对比



- ▶ 八线程曾塌陷至 3 个 A76、A55 全程空闲，根因为唤醒聚簇
- ▶ 对齐 Linux
select_idle_sibling: 仅 occ==0 时保留 last_cpu，否则选最空闲核，驻留随即均匀
- ▶ wake_affine 将 m1t4 唤醒 p50 由 1042 降至 8 μ s，接近 Linux 6 μ s，判定持平
- ▶ 出厂启用 idle-poll 后，并发工作线程经 select_idle_sibling 铺开，schbench RPS 339 反超 Linux 199 (1.71 $\times$)

正确性：页表与 TLB 修复的波及面



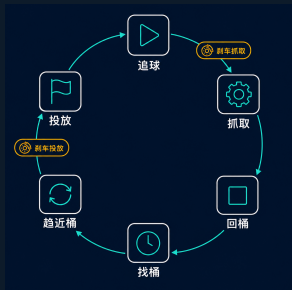
- ▶ 提升大页、跳过失效、引入 2 MB 块时，多代理对抗审查（每项 7 到 26 人）发现一批潜伏的 SMP 与 UAF 缺陷
- ▶ 主要一处：not-present 巨页块被误当页表遍历，unmap 时重复释放与 COW 共享帧构成 UAF；以 GenericPTE::is_table() 统一判定修复
- ▶ aarch64 侧修复 PTE 写序缺 dsb ishst、本地 vmalle1 应为广播、拆分对 OOM 不原子（波及面见左图）

经验证暂不适用当前平台的尝试

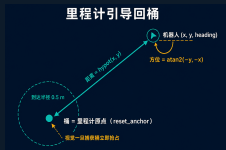
尝试	现象	为何无收益或回退
全量 work-stealing 均 衡器	单线程 350→161 (0.46×)，饱和无收益	七个空闲核反复迁移唯一工作线程，无核心保持 缓存热度
自适应 haltpoll	未稳定优于固定 50 μs	停机时长信号受 SGI-WFI 交互干扰而失真
early-MMU 启动重排	4.446 vs 4.46 s	低频启动窗口受 CPU 制约，缓存无从节省
第三 NPU 核	由两核到三核无收益	该模型约占一个核，增配核心无法占满

多数尝试在更换硬件类型或补足前置条件后仍有望见效，适用边界已随各条列出。

机器人拾球应用：状态机与里程计回桶



- ▶ 应用为一份 C++ 状态机，主机 dry-run 与板端同源，仅替换电机与机械臂后端
- ▶ 六状态拾球闭环（追球、抓取、回桶、寻桶、趋近、投放）纯逻辑非阻塞，定时时相由单调时钟推进而与相机 FPS 解耦
- ▶ 里程计引导回桶：投放时 `reset_anchor` 把桶设为原点，航位推算按 `distance` 与 `bearing` 回锚点，视觉一旦捕获桶即抢占
- ▶ 慢速轮速遥测移出控制回路，由后台线程异步更新位姿快照供每轮读取



StarryOS / RK3588 关键里程碑，日期为该能力落地的代表性节点

- ▶ 决赛约一个月 (07-01 至 08 月中), 三名队员协作, 各项以真板验证为准; 左图为七周里程碑, 自 perf 多核起至页表正确性与原子 PR
- ▶ 落地并验证: PMU perf、QEMU RKNPU、SMP 分布、DVFS 环长、占用调度、THP 首触、CLONE_VM 快路径、系统调用下降、TTFI 领先
- ▶ 三项如实回退或受限制: 工作窃取均衡器抖动回退、early-MMU 无收益回退、DDR 变频受主线固件限制

队员分工 (Team Posad, 三名成员)

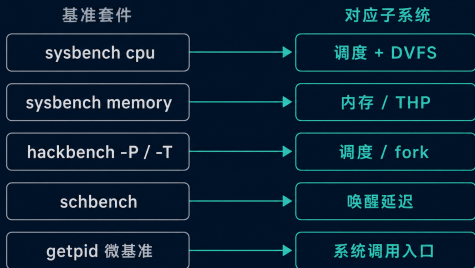
成员	负责模块与交付物
洪世金 (队长)	perf 观测 (#1395/#1577)、调度器与大小核 (#1495/#1656)、DVFS 与 cpufreq (#1657)、内存与 THP、页表与 TLB 正确性、RGA2 (#1388) 与 JPU (#1456) 驱动、原生显示栈 VOP2 / HDMI-QP / HDPTX、sysbench (#1658)、报告撰写与整体协调
王乙凡	QEMU RKNPU 功能级模拟器: 全流水线 + IOMMU + INT8/FP 执行 (qemu#19, 106 qtests), 多核协同 (qemu#26); 机器人拾球应用 (与徐子航共同)
徐子航	板级 I/O 与网络: UART (#1921)、USB tty (#1378)、PWM (#1468)、rtnetlink 与 SSH (#1497); UVC 相机 (#1924/#1961)、reboot (#1358)、GEM cache mmap (#1364); 机器人拾球应用 (与王乙凡共同)

目标完成情况

目标域	结果 (StarryOS 优化前 → 后, 对照 Linux)	状态
观测 perf/fttrace	硬件 PMU perf 与 fttrace 上板, big.LITTLE 全量通过	已达
调度与大小核	cpu 八线程 160→ 4987 ev/s, 为 Linux 5222 的 0.96×	已达
系统调用入口	getpid 1200→ 431 ns, 为 Linux 167 ns 的 2.6× (由 7× 收窄)	已达
内存与分页	THP first-touch 0.8 到 1.3→ 0.030 s, Linux 0.086 s, 快 2.87× (领先)	已达
频率电压	CPU 调频调压经 PVTPLL 环长对齐 Linux (逐核持平); 内存写 15040→ 34330 MiB/s (0.61×, DDR 变频属固件外因)	已达
启动 TTFI	19.22→ 9.83 s, 优于 Linux 11.28 s (领先)	已达
显示栈	VOP2 + HDMI-QP + HDPTX 上板点亮, ROPLL 锁定验证通过	已达
QEMU NPU 模型	RKNPU 功能级仿真合入, 106 个 qtest	已达
机器人应用	网球拾取机器人六状态闭环在 RK3588 真机完整运行, 感知到运动控制全链路	已达

基准套件与测量口径

基准套件与子系统对照



每项基准均在同一开发板上对同一 Linux 基线独立复测

- ▶ 每个子系统采用各自业界基准，各配一条同机同频 Linux 基线，两侧运行同一份 ELF
- ▶ cpu 考察调度与 DVFS，memory 考察分页与 THP，hackbench 考察 fork 与并发，schbench 考察唤醒延迟，getpid 考察系统调用入口
- ▶ 每点取 $N \geq 5$ 的 p50，两次读数漂移超 $\pm 2\%$ 判为噪声重测
- ▶ 主线用累计阶梯逐层叠加，相互影响者再单独正交消融

基础版本与增量贡献

PR 归属统计：子系统 × 贡献者

子系统	系统内核侧	仿真与板级侧
perf	■ #1395 ○ #1577 #1601 #1602 #1603	-
调度	■ #1495 ○ #1656 #1658	-
DVFS	■ #1657	-
内存	■ #1991 #1992 #1993 #2000 ○ #1990 #1994 #1996 #2011	-
正确性	○ #2008 #2009	-
驱动 RGA/JPU	■ #1388 #1456	-
板级 I/O	-	■ #1358 #1378 #1921 #1922 #1468 ■ #1924 #1961 #1497 #1963
QEMU-NPU	-	■ processmission/qemu#19 #26 ○ 逐推理计数
机器人	-	- 会话中（未开 PR）

- ▶ 基线冻结于 rcore-os/tgoskits 的 dev 分支 commit 73409e079，每处原子改动以此为起点派生为一个 PR
- ▶ 拆分至可独立回归的粒度，使每条改动可对照固定基线核对得失，评审仅需关注单一逻辑，触及 SMP 的改动另配多代理对抗审查
- ▶ 累计 90 个 PR 已合并，覆盖观测、调度、内存、DVFS、网络、驱动与外设初始化；19 个在审，含多核 perf 四件与调度分布（#1656，把八线程由平线 361 抬到 4987 ev/s）、COW 与 THP 一组

与类似工作的对比

- ▶ 多数 Rust OS 工作在 QEMU 或综合基准上自证，少有在真实边缘 SoC 上与同机 Linux 运行同一份 ABI 可执行文件的逐轴对照
- ▶ 本工作在 RK3588 真机上以同一份 ELF、同机同频、 $N \geq 5$ 的 p50 对照 Linux，覆盖调度、系统调用、内存、频率、启动五个轴
- ▶ 加速器侧将闭源 RKNPU 实现为功能级 QEMU 模型（106 qtests），使 NPU 驱动与模型优化脱离板子，属可复用产物
- ▶ 定调诚实：给出的是对等与少数领先，并如实标注外因（DDR 固件）与后续优化方向（系统调用入口、锁竞争）